ICT103 Programming

Topic 9

Inheritance, Polymorphism and Exceptions

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 8:

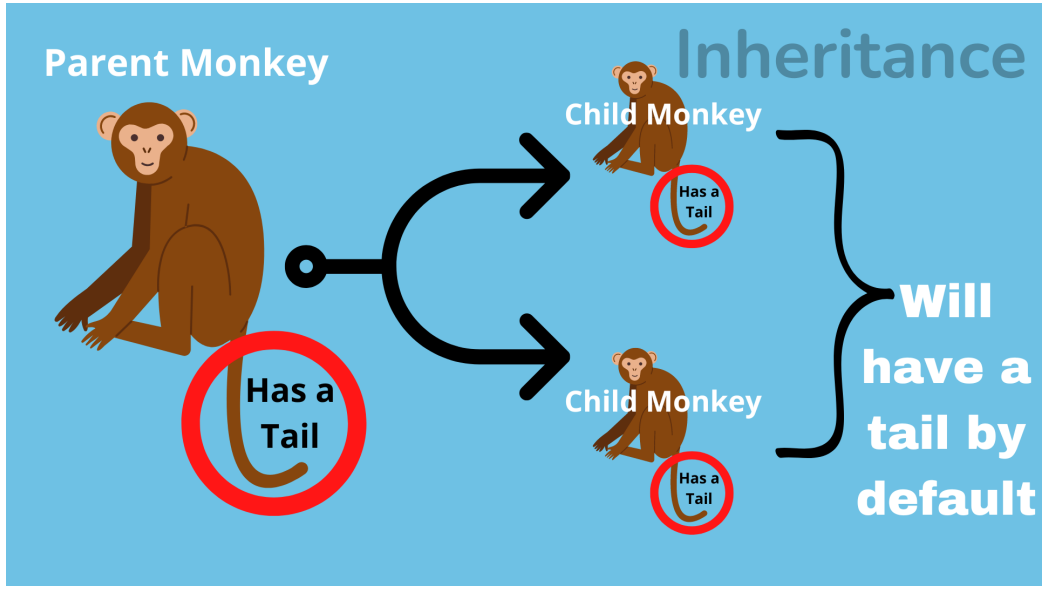
- Understand methods and their purpose
- Explain built-in overriding
- Use of Constructors in OOP
- Identify the visibility of variables in OOP

Week 9: Lesson Learning Outcomes

- Explain the role of inheritance in building a class hierarchy
- Explore the types of inheritance and necessary components
- Explore how polymorphism is used in OOP
- Dealing with Exceptions
- Using Exceptions

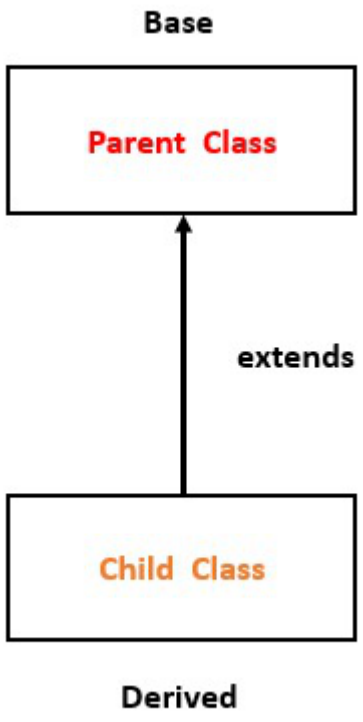
Inheritance

- Inheritance is a mechanism to create a new class from an existing class.
- The new class (child) inherits attributes and methods of the existing class (parent)
- **Parent class** (a.k.a. base class, super class): the top class in the hierarchy
- **Child class** (a.k.a. derived class): a class that inherits the behavior and data members from the parent class



Benefits of Inheritance

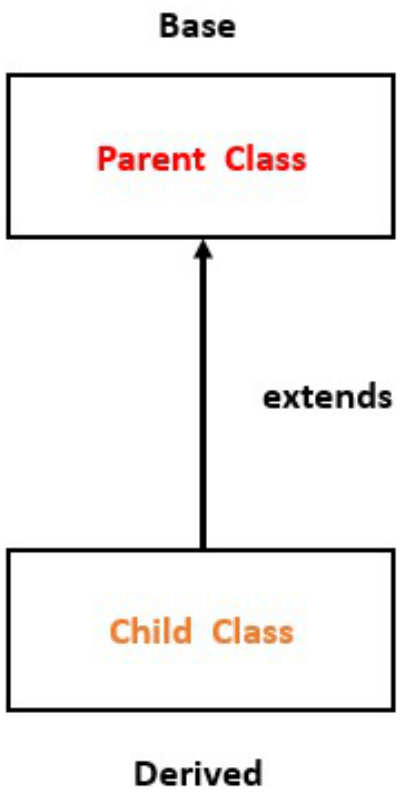
- **Code Reuse**
  - Avoid rewriting the same methods
- **Logical Hierarchy**
  - Represents real-world relationships
- **Extensibility**
  - New features can be added in child classes without changing the parent
- **Maintainability**
  - Centralised changes in parent class affect all children



Inheritance Syntax

```
class ParentClassName:
    # Body of Parent Class

class ChildClassName(ParentClassName):
    # Body of Child Class
```

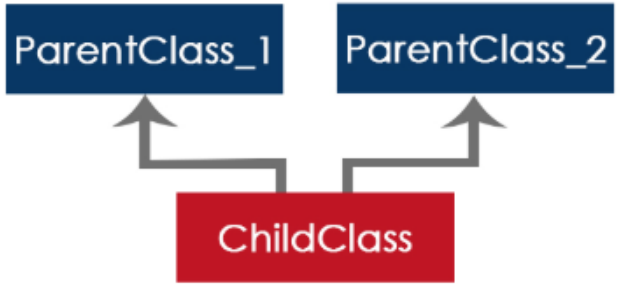


- The inheritance relationship is represented by an arrow pointing from **Derived Class(Child Class)** to **Base Class(Parent Class)**.
- The extends keyword denotes that the **Child Class** is inherited or derived from **Parent Class**.

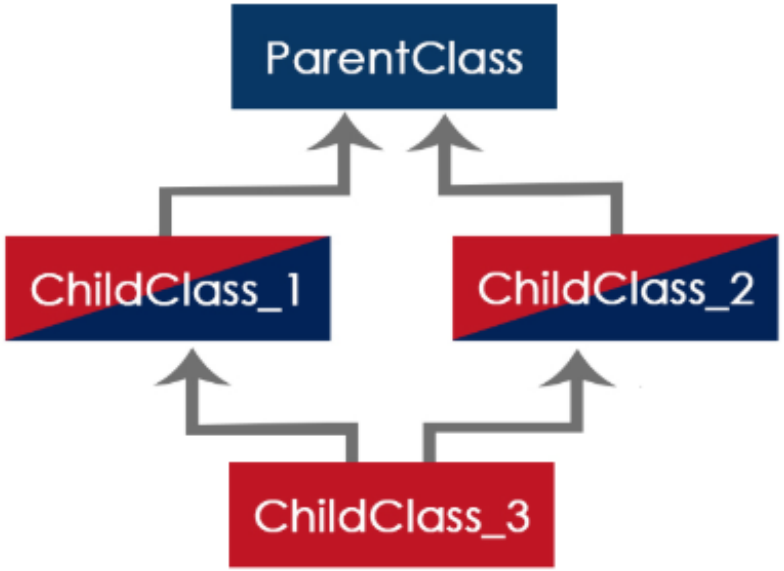
Types of Inheritance

Simple Inheritance

Multiple Inheritance

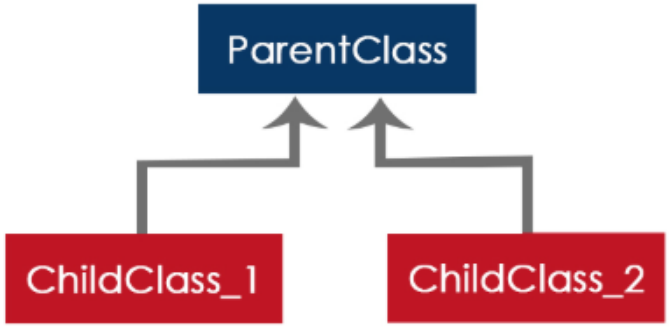


Hybrid Inheritance

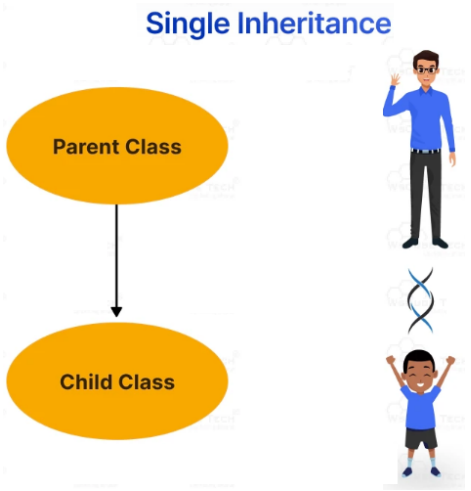


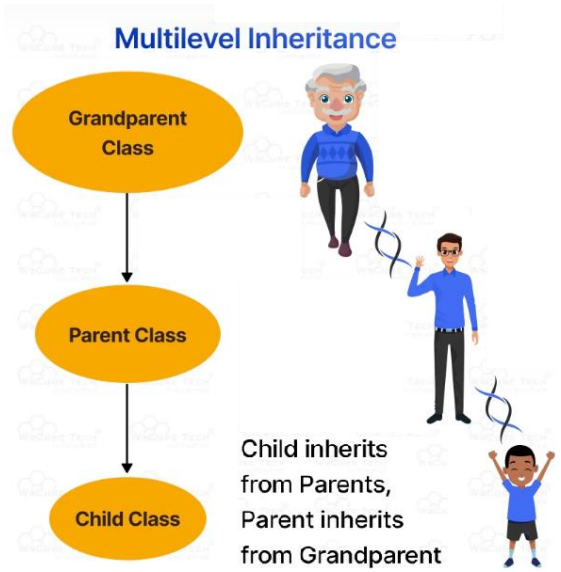
Multi Level Inheritance

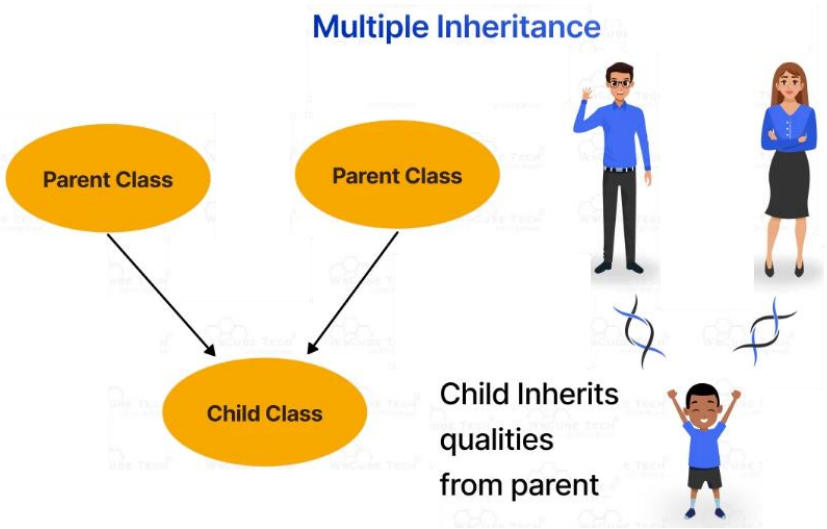
Hierarchical Inheritance

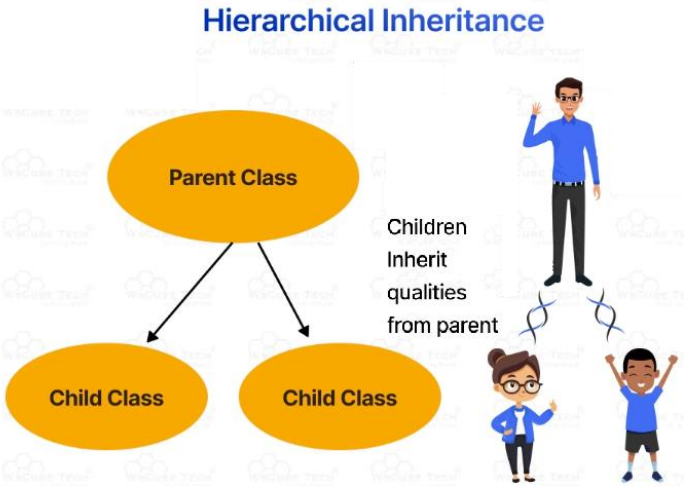


Examples of Inheritance









Use of super() function

- A super() allows a child class to access methods or attributes of it parent class.
- Useful when you override but still want the original parent behaviour.
- It is always used inside of the child class.
- It is used to avoid rewriting code and ensure the parent class is properly initialized.

```python
class Animal:
    def sound(self):
        print("Some sound from Animal")

class Dog(Animal):
    def sound(self):
        super().sound()  # call parent method
        print("Bark from Dog")

d = Dog()
d.sound()
```

Output

```
Some sound from Animal
Bark from Dog
```

Method Overriding

- Allows a child class to provide its own implementation of a method that is already inherited from one of its ancestors
  - Useful when a method must be included in child class but with different implementation
- Overridden methods should be called directly because the super() method will always choose the first version of the method in the Parent class

- Overriding is a type of Polymorphism in OOP

Polymorphism Basics

- Means “many forms”
- Same method name, different behaviour depending on object.
- Allows flexible and reusable code
- Python supports polymorphism via:
  - Method overriding (inheritance-based)
  - Duck typing (dynamic typing)
  - Built-in polymorphic functions

Method Overriding (OOP Polymorphism)

```python
class Animal:
    def sound(self):
        print("Some sound from Animal")

class Dog(Animal):
    def sound(self):
        super().sound()  # call parent method
        print("Bark from Dog")

d = Dog()
d.sound()
```

Output

```
Some sound from Animal
Bark from Dog
```

- Shows same method name, different behavior in child class

Duck Typing

```python
class Bird:
    def fly(self):
        print("fly with wings")

class Airplane:
    def fly(self):
        print("fly with fuel")

class Fish:
    def swim(self):
        print("fish swim in sea")

# Attributes having same name are
# considered as duck typing
for obj in Bird(), Airplane(), Fish():
    obj.fly()
```

Output

```
fly with wings
fly with fuel
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 20, in <module>
AttributeError: 'Fish' object has no attribute 'fly'
```

- "if it walks like a duck, quacks like a duck, then it's a duck."
- Used in dynamic languages like Python, Perl, Ruby, Javascript, etc
- It does not check the types at all, instead it checks the presence of a given method or attribute.

Built-in Polymorphism

```python
print(len("Hello"))  # 5
print(len([1,2,3])) # 3
print(2 + 3)         # 5
print("Hi" + "Bye")  # HiBye
```

Output

```
5
3
5
HiBye
```

- **len()** works on strings, lists, tuples
- Operators like **+** work on numbers and strings

Defining Exceptions

- An unexpected event during program execution

- Difference:
  - Error ⟶ syntax or compile-time problem
  - Exception ⟶ runtime problem, can be handled

```
1/0  # ZeroDivisionError
```

Output

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 1, in <module>
ZeroDivisionError: division by zero
```

Why handle exceptions?

- Prevent program crash
- Provide meaningful error messages
- Improve program reliability
- Maintain smooth user experience

Exception Types

- Built-in Exceptions:
  - Pre-defined exceptions in Python for common errors.

- User-defined Exceptions:
  - Custom exceptions created by programmers using `raise` and a new class.
  - Purpose: to enforce custom business rules or specific checks.

```python
class NegativeError(Exception):
    pass

x = -5
if x < 0:
    raise NegativeError("Negative numbers not allowed")
```

Output

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 6, in <module>
NegativeError: Negative numbers not allowed
```

Common Exceptions

Type	Situation	Example
IndexError	An index is outside an appropriate range	numbers = [] numbers[2]
TypeError	A function or method is given an argument that has an incorrect type	square_root("cat") set_name(5)
ValueError	A function or method is given an argument that has an incorrect value	square_root(-5) set_age(-12)
ZeroDivisionError	An attempt to divide by zero was made	x = 4 / 0

```
x = int("abc")  # ValueError
y = 10 / 0      # ZeroDivisionError
```

Output

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'abc'
```

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 2, in <module>
ZeroDivisionError: division by zero
```

Dealing with Exceptions

- Generally, to work with exceptions, programs **try** to do something and catch any exceptions that were thrown in the code

- **Keywords:**
  - Try: code that may raise an exception
  - Except: handle the exception
  - Else: runs if no exception
  - Finally: always runs, for cleanup
  - Raise: explicitly trigger an exception

Example

```python
try:
    x = int(input("Enter a number: "))
    print(10 / x)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Invalid input")
else:
    print("Success")
finally:
    print("End of program")
```

Output

```
Enter a number: 4
2.5
Success
End of program
```

Output

```
Enter a number: 0
Cannot divide by zero
End of program
```

Output

```
Enter a number: d
Invalid input
End of program
```

Do’s and Don’ts

- **Best Practices**
  - Catch specific exceptions (ValueError, ZeroDivisionError)
  - Keep try blocks minimal (only wrap risky code)
  - Use finally for cleanup (close files, release resources)
  - Provide clear error messages
  - Use exceptions fro unexpected errors, not for normal flow

```python
# Good: Catching the right exception
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

- **Common Mistakes**
  - Using bare except: ⟶ hides all errors
  - Ignoring exceptions (empty except or pass)
  - Catch the wrong exception type
  - Overusing exceptions instead of simple checks

```python
# Bad: Catching the wrong exception
try:
    result = 10 / 0
except ValueError:       # wrong type, won't catch
    print("Invalid value")
```

Next Week – What's Coming Up?

- What is file handling?
- Explain the use of I/O functions
- Explain file processing operations
- Understand how to read, close or create a new file
- Define `os` module

CIHE

CIHE Australia